Chapter : 7

Multithreaded Programming

7.0 OBJECTIVES

Dear Students,

After studying this chapter you are able to

- discuss what is multithreaded programming
- explain the Thread class in Java
- explain to create a thread and multiple threads
- discribe the methods isAlive() and join()
- discuss how to set priority of threads
- to get an introduction of synchronisation
- to get a brief overview of interthread communication

7.1 INTRODUCTION

Java provides built in support for multithreaded programming. A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a **thread**, and each thread defines a separate path of execution.

Thus, multithreading is a specialised form of multitasking. We have already studied that a process is a program in execution. This means that process based multitasking allows the computer to run two or more programs concurrently. In process based multitasking, a program is the smallest unit of code that can be dispatched by the scheduler. In thread based multitasking environment, the thread is the smallest unit of dispatchable code. Therefore a single program can perform two or more tasks concurrently. Multitasking threads require less overhead as compared to multitasking processes. Processes are heavyweight and require their own separate address spaces. Threads on the other hand are lightweight. They share the same address space. Interthread communication is inexpensive as compared to interprocess communication which is expensive and limited.

Multithreading enables you to write very efficient programs which make maximum use of CPU, because idle time can be kept at a minimum. This is very important for the interactive, networked environment in which Java works.

Threads in Java exist in several states. A thread can be **running**. It can be **ready to run** as soon as it gets the CPU time. A thread which is running can be **suspended**, and it temperorily suspends its activity. A suspended thread can be **resumed**, and it is allowed to pick up from where it left off. A thread can be **blocked** when waiting for a resource. At any time, a thread can be **terminated**, which halts its execution immediately. Once terminated a thread cannot be resumed.

Thread priorities : Java assigns to each thread a priority. The priority determines how that thread should be treated with respect to others. Thread priorities are integers which specify the relative priority of one thread to another. The priority of a thread is used to decide when to switch from one running thread to the next. This is called as *context switch*.

Messaging : Once a program has been divided into separate threads, it is necessary to define how they will communicate with each other. Java provides a way for two or more threads to communicate with each other via call to predefined methods that all objects have.

7.1 Check Your Progress.

1. Fill in the blanks.

a) Multithreading is a specialised form of ..... .

b) The ..... determines how a thread should betreated with respect to others.

c) ..... are heavyweight, whereas ..... are lightweight.

2. Write True or False.

a) Java provides built in support for multithreaded programming.

b) It is not possibleto temperorily suspend a thread which is running.

c) A thread once terminated can be resumed.

7.2 THE THREAD CLASS

Java's multithreading system is built upon the **Thread** class, its methods and the **Runnable** interface. **Thread** encapsulates a thread of execution. To create a new thread, your program should either extend the **Thread** or implement the **Runnable** interface.

The Main Thread : When a Java program starts up, one thread begins running immediately. This thread is usually called the main thread of the program since it is the one that is executed when the program begins. The main thread has the following features :

- It is the thread from which other child threads are spawned.
- It must be the last thread to finish execution. When the main thread stops, your program terminates.

The main thread is automatically created when the program starts. It can be controlled through a **Thread** object. You have to obtain a reference to it by calling the method **currentThread()** of the **Thread** class. Its general form is :

static Thread currentThread()

This method is **static** and returns a reference to the thread in which it is called. With this you can obtain a reference to the main thread and can then control it like any other thread.

The following example illustrates :

```
class ThreadDemo {
                    public static void main(String args[ ]) {
                        Thread th = Thread.currentThread();
                        System.out.println("Current thread is : " + th);
                        th.setName("MyThread");
                        System.out.println("After chaging name to
MyThread : " + th);
                    }
}
```

The output of the program is :

Current Thread : Thread[main, 5, main]

After Changing name to MyThread : Thread[MyThread, 5, main]

Here we obtain a reference to the current thread with the **currentThread()** method and store this reference in a **Thread** variable th. With the **println()** statement you display the information about the thread. As you can see in the output, by default the name of the thread is main, its priority is 5 and main is also the name of the group to which this thread belongs. A thread group is a data structure that controls the state of a collection of threads as a whole. The **setName()** method can be used to change the internal name of the thread. Then again th is output. The new name of the thread is now displayed.

sleep() is a method defined by **Thread** which causes the thread from which it is called to suspend execution for the specified period of milliseconds. Its general form is :

static void sleep(long *milliseconds*) throws InterruptedException

milliseconds specifies the number of milliseconds to suspend the thread. This method may throw an **InterruptedException**.

With the **setName()** method you can set the name of a thread. Similarly by using the **getName()** method you can obtain the name of a thread. These methods are :

final void setName(String *threadName*)

final String getName()

threadName specifies the name of the thread.

7.2 Check Your Progress.

1. Match the following :

	Column A	**Column B**
a)	main thread	i) used to obtain name of a thread
b)	currentThread()	ii) used to set name of a thread
c)	sleep()	iii) used to obtain reference to current thread
d)	setName()	iv) last thread to finish execution
e)	getName()	v) causes the thread from which it is called to suspend execution

7.3 CREATING A THREAD

You can create a thread by creating an instance of type **Thread**. This can be done in two ways :

- Implement the **Runnable** interface
- Extend the **Thread** class itself.

7.3.1 Implementing Runnable

You create a class that implements **Runnable** interface to create a thread. You can construct a thread on any object that implements **Runnable**. In order to implement **Runnable**, the class needs to implement only one method **run()**. Its form is :

public void run()

Inside **run()**, you define the code which constitutes the new thread. Remember that **run()** can call other methods, use other classes and also declare variables just like the main thread can. The only difference is that **run()** establishes the entry point for another concurrent thread of execution within your program. This thread ends when **run()** returns.

Once you create a class that implements **Runnable**, you instantiate an

object of type **Thread** from within the class. The constructor for **Thread** which we shall use is :

Thread(Runnable *threadObj*, String *threadName*)

threadObj is an instance of a class that implements the **Runnable** interface and this defines where the execution of the thread will begin. *threadName* is used to specify the name of the new thread.

A new thread does not start running, until you call its **start()** method. This method is declared within **Thread**. **start()** executes a call to **run()** and has the following form :

void start()

The following example illustrates how to create a new thread and the use of the **start()** method :

```
class NewThread implements Runnable {
                    Thread t ;
                    NewThread() {
                        t = new Thread(this, “Demo Thread”);
                        System.out.println(“Child Thread : “ + t);
                        t.start();
                    }
                    public void run() {
                        try {
                            for(i = 5; i > 0; i--) {
                                System.out.println(“Child
                            Thread : “ + i);
                            Thread.sleep(500);
                    }
    } catch(InterruptedException e) {

System.out.println(“Interrupted
Child Thread”);
                        }
                        System.out.println(“Exiting Child Thread”);
                    }
    }
    class ThreadDemo {
        public static void main(String args[ ]) {
                        new NewThread();
                        try {
```

```
                    for(int i = 5; i > 0 ; i--) {
                        System.out.println("In
                    Main Thread : " +i);
                        Thread.sleep(1000);
                    }
                } catch(InterruptedException e) {
                    System.out.println("Main Thread
                Interrupted");
                }
                System.out.println("Exiting Main Thread");
            }
        }
```

In this program we create a **Thread** object with the following statement :

```
t = new Thread(this, "Demo Thread");
```

Passing this as the first argument indicates that you want the new thread to call the **run()** method on this object. **start()** method starts the execution of thread which begins at the **run()** method. The child thread's **for** loop begins execution. After calling **start()**, NewThread's constructor returns to **main()**. When the main thread resumes, it enters its **for** loop. Both the threads continue running, sharing the CPU till their respective loops finish.

The output of the program is :

```
Child thread : Thread[Demo Thread, 5, main]
In Main Thread : 5
Child Thread : 5
Child Thread : 4
In Main Thread : 4
Child Thread : 3
Child Thread : 2
In Main Thread : 3
Child Thread : 1
Exiting Child Thread
In Main Thread : 2
In Main Thread : 1
Exiting Main Thread
```

Remember that in a multithreaded program, the main thread must be the last thread to finish running. In case the main thread finishes before a child thread has completed, then the Java run time system may hang. Therefore we make the main thread sleep for 1000 milliseconds between iterations and the child

thread for 500 milliseconds. This causes the child thread to terminate first.

7.3.2 Extending Thread

In the second method, you create a new class that extends **Thread** and then create an instance of that class. The extending class has to override the **run()** method which is the entry point for the new thread. It must also call the **start()** method to begin execution of the new thread. Let us rewrite the above program by extending **Thread** :

```
class NewThread extends Thread {
                    NewThread() {
                        super("Demo Thread");
                        System.out.println("Child Thread : " + this);
                        start();
                    }
                    public void run() {
                        try {
                            for(int i = 5; i > 0; i--) {
                                System.out.println("In
                            Child Thread : " + i);
                                Thread.sleep(500);
                            }
                            } catch(InterruptedException e) {
                                System.out.println("Child
                            Interrupted");
                            }
                        }
                        System.out.println("Exiting  child thread");
                    }
}
class ExtendThread {
                    public static void main(String args[ ]) {
                        new NewThread();
                        try {
                            for(int i = 5; i > 0 ; i--) {
                                System.out.println("In
                            Main Thread : " +i);
                                Thread.sleep(1000);
                            }
```

```
                } catch(InterruptedException e) {
                        System.out.println("Main Thread
                Interrupted");
                }
                System.out.println("Exiting Main Thread");
        }
}
```

This program generates the same output as the previous one. We make use of **super()** inside NewThread.

7.3 Check Your Progress.

1. Answer the following.

a) List the methods which can be used to create a thread by creating an instance of type **Thread**.

_____

_____

b) Which is the method used to start running a thread? What is its form?

_____

_____

7.4 CREATING MULTIPLE THREADS

In the following program we shall see how to create multiple child threads.

```
class NewThread implements Runnable {
        String name;
        Thread t;
        NewThread(String threadname) {
                name = threadname;
                t = new Thread(this, name);
                System.out.println("New thread : " + t);
                t.start();
        }
        public void run() {
                try {
                        for(int i = 5; i > 0; i--) {
                                System.out.println(name + ":" +
```

```
                    i);
                        Thread.sleep(1000);
                    }
                } catch(InterruptedException e) {
                        System.out.println(name +
                    "interrupted");
                    }
                System.out.println("Exiting" + name +
            "thread");
            }
}
class MultipleThread {
            public static void main(String args[ ]) {
                new NewThread("First");
                new NewThread("Second");
                new NewThread("Third");
                try {
                    Thread.sleep(10000);
                } catch(InterruptedException e) {
                    System.out.println("Main thread
                interrupted");
                }
                System.out.println("Exiting main thread");
            }
}
```

Run the program to determine what output it produces. Once started all the three child threads share the same CPU. Note that we have used the **sleep()** method in **main()** where we put the main thread to sleep for 10000 milliseconds. This ensures that all the child threads finish before main thread which should finish last.

7.5 ISALIVE() AND JOIN()

In all the preceding examples we called the **sleep()** method in **main()** in order to ensure that the main thread finishes last. However the **Thread** class defines a method to determine whether a thread has finished.

This method is **isAlive()** and its general form is :

final boolean isAlive()

If the thread upon which the method is called is still the running the method returns **true** else it returns a **false**.

One more method which is commonly used to wait for a thread to finish is **join()**. Its form is :

final void join() throws InterruptedException

This method waits until the thread on which it is called terminates. Other forms of **join()** are also there where you can specify the maximum amount of time that you want to wait for the specified thread to terminate.

Let us modify the previous program to make use of the **join()** method and ensure that main thread finishes last.

```
class NewThread implements Runnable {
    String name;
    Thread t;
    NewThread(String threadname) {
        name = threadname;
        t = new Thread(this, name);
        System.out.println("New thread name : " + t);
        t.start();
    }
    public void run() {
        try {
            for( int i = 3; i > 0; i --) {
                System.out.println(name + ":" +
                i);
                Thread.sleep(1000);
            }
        } catch(InterruptedException e) {
            System.out.println("Thread " + name +
            "interrupted");
            }
        System.out.println("Exiting " + name);
    }
}
class JoinDemo {
    public static void main(String args[ ]) {
        NewThread t1 = new NewThread("First");
        NewThread t2 = new NewThread("Second");
        NewThread t3 = new NewThread("Third");
        System.out.println("First thread is alive" +
    t1.t.isAlive());
```

```
                System.out.println("Second thread is alive"
+ t2.t.isAlive());
                System.out.println("Third thread is alive" +
            t3.t.isAlive());
                try {
                        System.out.println("Using join to
                wait for threads to finish");
                        t1.t.join();
                        t2.t.join();
                        t3.t.join();
                } catch(InterruptedException e) {
                        System.out.println("Main thread
                interrupted");
                }
                System.out.println("First thread is alive " +
            t1.t.isAlive());
                System.out.println("Second thread is alive "
+ t2.t.isAlive());
                System.out.println("Third thread is alive " +
t3.t.isAlive());
            }
}
```

Sample output of the program is :

```
New thread : Thread[First, 5, main]
New thread : Thread[Second, 5, main]
New thread : Thread[Third, 5, main]
First thread is alive : true
Second thread is alive : true
Third thread is alive : true
First : 3
Second : 3
Third : 3
First : 2
Second : 2
Third : 2
First : 1
Second : 1
Third : 1
```

```
Exiting First
Exiting Third
Exiting Second
First thread is alive : false
Second thread is alive : false
Third thread is alive : false
```

7.4 & 7.5 Check Your Progress.

1. Fill in the blanks.

a) The ..... thread should finish last when multiple threads have been created.

b) Thread class defines the ..... method to determine whether a thread has finished.

c) The method which is commonly used to wait for a thread to finish is ..... .

7.6 THREAD PRIORITIES

Java assigns to each thread a priority. The priority determines how that thread should be treated with respect to others. Thread priorities are integers which specify the relative priority of one thread to another. The priority of a thread is used to decide when to switch from one running thread to the next. This is called as **context switch**. Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run. Theorotically higher priority threads get more CPU time than the threads with lower priority. However in practice, the amount of CPU time that a thread gets depends upon a number of factors. The **setPriority()** method, which is a member of the **Thread** class is used to set the priority of a thread. Its general form is :

final void setPriority(int level)

where level specifies the new priority setting for the calling thread. The value of level should be within the range **MIN_PRIORITY** and **MAX_PRIORITY**. These values are 1 and 10 respectively. To return a thread to its default priority you use specify **NORM_PRIORITY** which is currently 5. These priorities are defined as final values in the **Thread** class.

The current priority can be obtained using the **getPriority()** method. Its form is :

final int getPriority()

It is important to note that implementations of Java have radically different behaviour when it comes to scheduling. Some versions like Windows works as you would expect, other versions may work quite differently.

Let us write a program to set two threads to different priorities. The instance hi is set to a higher priortiy than **NORM_PRIORITY** and lo is set to a priority less

than the **NORM_PRIORITY**. Both the threads will increment the values of their counter within the alloted CPU time.

```
class Demo implements Runnable {
                int i = 0;
                Thread t;
                private volatile boolean running = true;
                public Demo(int p) {
                        t = new Thread(this);
                        t.setPriority(p);
                }
                public void run() {
                        while(running)
                                i++;
                }
                public void stop() {
                        running = false;
                }
                public void start() {
                        t.start();
                }
}
class TPriority {
                public static void main(String args[ ]) {

Thread.currentThread().setPriority(Thread.MAX_PRIORITY);
                        Demo hi = new
                Demo(Thread.NORM_PRIORITY + 2);
                        Demo lo = new
                Demo(Thread.NORM_PRIORITY - 2);
                        lo.start();
                        hi.start();
                        try {
                                Thread.sleep(10000);
                        } catch(InterruptedException e) {
                                System.out.println("Main thread
                        interrupted");
                        }
```

```
                lo.stop();
                hi.stop();
                try {
                        hi.t.join();
                        lo.t.join();
                } catch(InterruptedException e) {
                System.out.println("InterruptedException
        caught");
                }
                System.out.println("Low priority Thread : " +
lo.i);
                System.out.println("High priority Thread :" +
hi.i);
        }
}
```

The output of this program will vary from machine to machine and will also depend upon a number of factors like the CPU speed of the machine on which it is running and the other tasks that may be running on the system. In this example, running is preceded by the keyword **volatile**. **volatile** ensures that the value of running is examined each the the loop iterates. If the **volatile** keyword is not used, Java is free to optimise the loop as follows : It stores the value of running in a register of the CPU and does not necessarily check it at every iteration.

7.6 Check Your Progress.

1. Write True or False.

a) A priority is assigned to each thread by Java.

b) Thread priorities are integer values.

c) It is not possible to obtain the current priority of a thread.

d) All implementations of Java have similar behaviour when it comes to scheduling.

7.7 SYNCHRONIZATION

It is possible that two or more threads are required to access a common resource. Therefore there has to be some way to ensure that the resource would be used by only one thread at a time. The process by which this is achieved is called as synchronization. A monitor is an object that is used as a mutually exclusive lock or mutex. Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have entered the monitor. All the other threads which are attempting to enter the locked monitor will be suspended until the first thread exits the monitor. These threads are said to be waiting for the monitor. It is also possible for a thread which owns a monitor to re-enter the same monitor.

In order to synchronize the code, Java provides two ways, both of which make use of the **synchronized** keyword. Let us study these methods.

7.7.1 Synchronized methods

In Java, all objects have their own implicit monitor associated with them. In order to enter an object's monitor, you just call a method which has been modified by the keyword **synchronized**.

When a thread is inside a synchronized method, all the other threads which try to call that method or any other synchronized method, on the same instance have to wait. When the owner of the monitor returns from the synchronized method, it exits the monitor and relinquishes the control of the object to the next waiting thread.

In the following example, we shall create a synchronized method in a class. This program has three classes. The first class called First has a method **display()** which outputs a string. After the method is called, we put the thread to sleep for one second. The second class, Second has a constructor which takes a reference to an instance of First and a String. This constructor creates a new thread which will call the run method of this object. The **run()** method of Second, calls the display() method on the instance of First by passing the message. The last class is the SynchDemo which creates an instance of First, and three instances of Second and passes a String message. However, the same instance of First is passed to Second.

In this situation, if you do not synchroize the display() method, it allows the execution to switch to another thread by calling **sleep()**. Therefore, there is nothing to stop all the three threads from calling the same method, on the same object at the same time. This is called as *race condition*, because the three threads are racing each other to complete the method. To overcome this, you have to serialize the access to display(). This implies that you restrict the access to only one thread at a time and therefore you precede the display() method with the **synchronized** keyword.

```
class First {
                    synchronized void display(String s) {
                        System.out.println(s);
                        try {
                            Thread.sleep(1000);
                        } catch(InterruptedException e) {
                            System.out.println("Interrupted");
                        }
                        System.out.println("***");
                    }
}
```

```
class Second implements Runnable {
        String s;
        First f;
        Thread t;
        public Second(First f1, String s1) {
                f = f1;
                s = s1;
                t = new Thread(this);
                f.start();
        }
        public void run() {
                t.display(s);
        }
}
class SyncDemo {
        public static void main(String args[ ]) {
                First f = new First();
                Second ob1 = new Second(f, “First”);
                Second ob2 = new Second(f, “Second”);
                Second ob3 = new Second(f, “Third”);
                try {
                        ob1.t.join();
                        ob2.t.join();
                        ob3.t.join();
                } catch(InterruptedException e) {
                        System.out.println(“Interrupted”);
                }
        }
}
```

As an exercise, first run this program without the use of the **synchronized** keyword with the display() method. You will obtain a mixed up output of the three message strings. Then run the same program by using the **synchronized** keyword.

Thus any time you have a method or a group of methods which manipulates the internal state of an object in a multithreaded situation, you have to use the **synchronized** keyword to prevent race conditions. Once a thread enters a synchronized method on an instance, no other thread can enter any other synchronized method on that instance. But remember, non synchronized methods

on that instance can still be called.

7.7.2 synchronized statement

There may be situations where you may want to synchronize access to objects of a class which has not been designed for multithreaded access i.e. the class does not have any synchronized methods. Also if a class has been created by a third party where you have no access to the source code, it is not possible for you to precede the methods of that class by the **synchronized** keyword. For this purpose, you put calls to the methods defined by this class inside a synchronized block. The generalised form of the synchronized statement is :

```
synchronized(object) {
                    // statements to be synchronized
}
```

where object is a reference to the object being synchronized. A synchronized block ensures that a call to a method which is a member of object will occur only after the current thread has successfully entered the object's monitor.

Let us modify the previous program itself, where the display() method is not preceded by the keyword **synchronized**, but the synchronized statement is used in the **run()** method of Second. This program will produce the same output. Each thread will wait for the previous thread to finish before proceeding.

```
class First {
                    void display(String s) {
                    System.out.println(s);
                    try {
                            Thread.sleep(1000);
                    } catch(InterruptedException e) {
                            System.out.println("Interrupted");
                    }
                    System.out.println("***");
                }
}
class Second implements Runnable {
                    String s;
                    First f;
                    Thread t;
                    public Second(First f1, String s1) {
                        f = f1;
                        s = s1;
                        t = new Thread(this);
```

```
                t.start();
        }
        public void run() {
                synchronized(f) {
                f.display(s);
                }
        }
}
class SyncDemo {
        public static void main(String args[ ]) {
                First f = new First();
                Second ob1 = new Second(f, “First”);
                Second ob2 = new Second(f, “Second”);
                Second ob3 = new Second(f, “Third”);
                try {
                        ob1.t.join();
                        ob2.t.join();
                        ob3.t.join();
                } catch(InterruptedException e) {
                        System.out.println(“Interrupted”);
                }
        }
}
```

7.7 Check Your Progress.

1. Answer the following.

a) What is synchronization?

_____

_____

b) What does a synchronized block ensure?

_____

_____

7.8 INTERTHREAD COMMUNICATION

Java includes interprocess communication mechanism via the **wait()**, **notify()** and **notifyAll()** methods. All these methods are implemented as

final methods in **Object** so all classes have them. All these methods can be called only from within a **synchronized** method. These methods are declared in **Object** as :

final void wait() throws InterruptedException

final void notify()

final void notifyAll()

wait() tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls **notify()**.

notify() wakes up the first thread that called **wait()** on the same object.

notifyAll() wakes up all the threads that called **wait()** on the same object. The thread with the highest priority will run first.

These methods have just been introduced to the student. A detailed study of interthread communiction via these methods is not included in the study material.

7.9 DEADLOCK

A deadlock situation occurs when two threads have a circular dependancy on a pair of synchronized objects. eg. Suppose one thread enters the monitor on object X and another thread enters the monitor on object Y. If the thread in X tries to call any synchronized method on Y, it will block. However, if the thread in Y tries to call any synchronized method on X, the thread waits forever, because to access X it will have to release its own lock on Y so that the first thread could complete.

7.10 SUSPENDING, RESUMING AND STOPPING THREADS

Earlier versions of Java, had the methods **suspend()**, **resume()** and **stop()** defined by **Thread** to control a thread. **suspend()** and **resume()** are methods which pause and restart the execution of a thread. However these methods have been deprecated by Java 2. Therefore, a thread should be designed in such a manner that **run()** periodically checks to determine whether that thread should suspend, resume or stop its own execution. To accomplish this normally a flag variable is established that indicates the execution state of the thread. As long as this flag is set to running, the **run()** method should continue to let the thread execute, if this variable is set to suspend, the thread must pause and if it is set to stop, the thread must terminate.

7.8 to 7.10 Check Your Progress.

1. Answer the following.

a) What is the use of the wait() method?

_____

_____

b) When does a deadlock situation occur?

_____

_____

c) How should a thread be designed in versions of Java, where the uspend(), resume() and stop methods have been deprecated?

_____

_____

7.11 SUMMARY

A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution.

Thread priority determines how that thread should be treated with respect to others. Once a program has been divided into separate threads, it is necessary to define how they will communicate with each other. Java provides a way for two or more threads to communicate with each other via call to predefined methods that all objects have.

Synchronization: It is possible that two or more threads are required to access a common resource. Therefore there has to be some way to ensure that the resource would be used by only one thread at a time. The process by which this is achieved is called as synchronization.

Source : *r4r.co.in* *(Link)*

7.12 CHECK YOUR PROGRESS - ANSWERS

7.1

1. a) multitasking

b) priority

c) Processes, threads

2. a) True

b) False

c) False

7.2

1. a) iv)

b) iii)

c) v)

d) ii)

e) i)

7.3

1. a) The ways used to create a thread by creating an instance of type **Thread** are : implement the **Runnable** interface and Extend the **Thread** class itself.

b) The **start()** method declared within **Thread** executes a call to **run()** and is used to make a thread run. Its form is : void start().

7.4 & 7.5

1. a) main

b) isAlive()

c) join()

7.6

1. a) True

b) True

c) False

d) False

7.7

1. a) The process by which it is ensured that a common resource required by two or more threads would be used by only one thread at a time is called as synchronization.

b) A synchronized block ensures that a call to a methodwhich is a member of object will occur only after the current thread has successfully entered the object's monitor.

7.8 to 7.10

1. a) The use of the **wait()** method is that it tells the calling thread to give up the monitor and go to sleep until someother thread enters the same monitor and calls **notify()**.

b) A deadlock situation occurs when two threads have a circular dependancy on a pair of synchronized objects.

c) A thread should be designed in such a manner that **run()** periodically checks to determine whether that thread should suspend, resume or stop its own execution in versions of Java where the suspend(), resume() and top() methods have been deprecated.

7.13 QUESTIONS FOR SELF - STUDY

1. Compare multiprogramming and multithreading.
2. Write short notes on the following :
   a) States of a thread
   b) Implementing Runnable
   c) Extending Thread
   d) Thread Priorities
   e) Deadlock
3. What is the main thread? Describe its features.
4. Describe the methods : currentThread() and sleep() alongwith their general forms.
5. Describe in brief the use of isAlive() and join() and write the general forms of these methods.
6. Describe synchronization. Describe the ways in which code can be synchronized in Java.
7. List and describe briefly the methods used for interthread communication.

7.14 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

❑ ❑ ❑

Notes

Notes

Chapter : 8

Strings

8.0 OBJECTIVES

Dear Students,

After studying this chapter you will able to :

- describe how to creat String objects and constructors of String
- discuss special String operations
- discuss the String comparison functions
- explain the character extraction functions
- describe the various functions to modify, compare and search strings.
- discuss what is meant by command line arguments.

8.1 INTRODUCTION

We have already studied that in Java a string is a sequence of characters.

Java implements strings as objects of type **String**. **String** objects can be constructed in a number of ways, therefore it is easy to obtain a string when needed. Java has a number of methods to perform various operations on strings like comparing strings, concatenating strings, changing case of letters within a string etc.

Remember that once a **String** object has been created, you cannot change the characters that comprise the string. This means that the contents of the **String** instance cannot be changed after it has been created. However, a variable declared as a **String** reference can be changed to point to some other **String** object anytime.

In those cases, that you require that a string is to be changed, there is an alternative class called **StringBuffer**, whose objects contain strings that can be modified after they are created. Both **String** and **StringBuffer** are defined in **java.lang**. This implies that they are automatically available to programs. Both are declared **final**, therefore they cannot be subclassed.

String represents fixed length, immutable character sequences. On the other hand, **StringBuffer** represents growable and writeable character sequences. **StringBuffer** may have characters and substrings inserted in the middle or appended at the end. **StringBuffer** will automatically grow to make room for such additions. We shall cover **String** and not address **StringBuffer** in this study material.

8.1 Check Your Progress.

1. Fill in the blanks.

a) Java implements strings as objects of type ..... .

b) The contents of the ..... instance cannot be changed after it has been created.

c) ..... is the class whose objects contain strings that can be modified after they are created.

8.2 THE STRING CONSTRUCTORS

Let us see the various constructors supported by the **String** class.

- To create an empty **String**, you call the default constructor eg.

  String s = new String();

  This will create an instance of **String** which has no characters.

- To create a **String** initialised by an array of characters we can use the following constructor :

  String(char *chars*[])

  eg. char chars[ ] = { ‘a’, ‘b’, ‘c’ };

  String s = new String(chars);

  This constructor will initialise s with the string “abc”.

- A subrange of a character array can also be specified as an initialiser for a

string. Its constructor is:

String(char *chars*[ ], int *startIndex*, int *numChars*)

startIndex specifies the index at which the subrange begins, and *numChars* specifies the number of characters to be used. eg.

```
char chars[ ] = {'a', 'b,' 'c', 'd', 'e' , 'f', 'g'};
String s = new String(chars, 3,2);
```

This set of statements will initialise s with the characters cd

You can construct a **String** object that contains the same character sequence as another **String** object. You use the following constructor for this purpose :

String(String *strObj*)

where *strObj* is a **String** object.

The following example will illustrate :

```
class StringDemo {
    public static void main(String args[ ]) {
            char c[ ] = {'a', 'b', 'c', 'd', 'e'};
            String s1 = new String(c);
            String s2 = new String(s1);
            System.out.println(s1);
            System.out.println(s2);
    }
}
```

The output of this program will be :

```
abcde
abcde
```

This shows that s1 and s2 both contain the same string.

We have already studied that **char** uses 16 bits to represent the Unicode character set. A typical format on the Internet however uses arrays of 8 bit bytes constructed from the ASCII character set. For this purpose the **String** class provides constructors which initialise a string when given a byte array. Their forms are :

String(byte *asciiChars*[ ])

String(byte *asciiChars*[ ], int *startIndex*, int *numChars*])

where *asciiChars* specifies the array of bytes. As you can see, the second form will allow you to specify a subrange. In both these constructors, the byte to character conversion is done by using the default character encoding of the platform.

The following example will illustrate the use of these constructors :

```
class Strings {
```

```
    public static void main(String args[ ])     {
        byte a[ ] = {65, 66, 67, 68, 69, 70, };
        String s1 = new String(a);
        System.out.println(s1);
        String s2 = new String(a, 2, 3);
        System.out.println(s2);
    }
}
```

The output of the program will be:

ABCDE

CDE

String length :

The length of the string is the number of characters it contains. The **length()** method is used to determine the length of the string. Its form is :

int length()

Thus it returns the value of type **int**.

eg. int i = s.length();

where s is an object of type **String**.

8.2 Check Your Progress.

1. Write the String constructors for the following.

a) To create an empty String : .....

b) To create a String initialised by an array of characters : .....

c) To initialise a string when given a byte array : .....

8.3 SPECIAL STRING OPERATIONS

Let us study some special string operations in this section.

8.3.1 String Literals

In the previous examples we saw how to create an instance of **String** from an array of characters with the use of the **new** operator. However, there is an easier and more convenient way to do this. You can use a string literal to initialise a **String** object. For each literal in your program, Java automatically constructs a **String** object.

eg. String s1 = “abc”;

will create s1 with contents abc.

A **String** object is created for every string literal, therefore you can use a string literal at any place where you can use a String object. eg. you can call methods directly on a quoted string as if it were an object reference. eg.

System.out.println(“abc”.length());

8.3.2 String Concatenation

Java does not allow operators to be applied to **String** objects. The only exception to this rule is the + operator, which concatenates two strings and produces a **String** object as the result. eg.

String s1 = “Good”;

String s2 = “Morning”;

String s3 = s1 + s2 + “to you”;

System.out.println(s3);

will output “Good Morning to you”

You can also concatenate strings with other data types. eg.

int time = 8;

String s1 = “It is “ + time + “o’clock”;

System.out.println(s1);

In this case, time is an **int** data type. The **int** value in time will automatically get converted into its string representation within a **String** object. The string will then be concatenated and the output will be:

It is 8 o’clock

8.3.3 toString()

When Java converts data into its string representation during concatenation, it does so by calling one of the overloaded versions of the string conversion method **valueOf()** defined by **String**. **valueOf()** is overloaded for all the simple types and for type **Object**. For the simple types **valueOf()** returns a string that contains the human readable equivalent of the value with which it is called. For objects, **valueOf()** calls the **toString()** method on the object.

Every class implements **toString()** because it is defined by **Object**. Many times you will want to override **toString()** and provide your own string representations. This is easily accomplished. The general form of **toString()** is :

String toString()

Thus just returning a **String** object that contains the human readable string which describes an object of your class is sufficient.

By overriding **toString()** for the classes that you create, you allow the resulting strings to be fully integrated into Java’s programming environment. Thus they can be used in the **print()** and **println()** statements and in concatenation. The following example illustrates :

```
class StringsDemo {
    int i, j;
    StringsDemo(int a, int b) {
```

```
                i = a;
                j = b;
        }
        public String toString() {
                return “Value of i “ + i + “and value of j “ + j;
        }
}
class DemotoString {
        public static void main(String args[]) {
                StringsDemo s = new StringsDemo(10,
100);
                System.out.println(s);
                String msg = “Automatic invokation of
String method : “ + s;
                System.out.println(msg);
        }
}
```

The output of this program is :

Value of i 10 and value of j 100

Automatic invokation of String method : Value of i 10 and value of j 100

Thus you can see that the method **toString()** is invoked automatically whenever you use an instance of StringsDemo in **println()**.

8.3 Check Your Progress.

1. Write True or False.

a) A string literal cannot be used to initialise a String object.

b) The + operator can be applied to String objects.

c) Every class implements toString().

8.4 EXTRACTING CHARACTERS

In this section let us see the different ways in which characters can be extracted from a **String** object.

8.4.1 charAt()

With this method you can directly refer to a single character. Its general form is :

char charAt(int *where*)

where is the index of the character you want to obtain. Remember that the value of where must not be negative and should specify a location within the string.

eg. char ch;

ch = “Mystring”.charAt(4) will assign the value 'r' to ch.

8.4.2 getChars() :

The general form of **getChars()** is :

void getChars(int *sourceStart,* int *sourceEnd*, char *target*[ ], int *targetStart*)

sourceStart specifies the index of the beginning of the substring, and *sourceEnd* specifies the index that is one past the end of the desired substring. This means that the substring will contain characters from *sourceStart* to *sourceEnd*-1. The receiving array is specified by *target*. The index at which the substring will be copied into target is specified by *targetStart*. It should be ensured that target is large enough to hold the specified number of characters.

The following example will demonstrate use of **getChars()**

```
class StringsDemo {
    public static void main(String args[]) {
            String s1 = “Extracting characters from
String”;
            start = 4;
            end = 10;
            char substring[] = new char[end - start];
            s1.getChars(start, end, substring, 0);
            System.out.println(“The extracted
characters : “);
            System.out.println(substring);
    }
}
```

The output of the program will be :

acting

8.4.3 getBytes() :

Is a variation of **getChars()**, in which the characters are stored in an array of bytes. This method uses the default character to byte conversions provided by the platform. Its simplest form is :

byte[ ] getBytes()

8.4.4 toCharArray() :

This method is used if you want to convert all the characters in a **String** object into a character array. The general form of this method is :

char[ ] toCharArray()

Note that this method returns an array of characters for the entire string.

8.4 Check Your Progress.

1. Match the following.

	Column A		**Column B**
a)	charAt()	i)	characters are stored in an array of bytes.
b)	getBytes()	ii)	Converts all the characters in a String object into a character array
c)	toCharArray()	iii)	can directly refer to a single character

8.5 COMPARING STRINGS

In this section let us study some method used to compare strings and substrings.

equals() : The general form of **equals()** is :

boolean equals(Object *str*)

str is the **String** object to be compared with the invoking **String** object. This method returns **true** if the strings are the same, i.e. they contain the same characters in the same order, else returns **false**.

equalsIgnoreCase() : With this method, a comparison which ignores the case can be performed Its general form is :

boolean equalsIgnoreCase(String *str*)

where *str* is the *String* object to be compared with the invoking *String* object. As in the case of **equals()**, it returns a **true** if the strings are the same, i.e. they contain the same characters in the same order, else returns **false**.

Let us see an illustration of the above methods :

```
class StringsDemo {
        public static void main(String args[ ]) {
                String s1 = “Good”;
                String s2 = “gOOd”;
                String s3 = “Good”;
                System.out.println(s1 + “ equals ”  + s3 + “---> “
+ s1.equals(s3));
                System.out.println(s1 + “ equals ”  + s2 + “-
--> “ + s1.equals(s2));
                System.out.println(s1+“
        equalsIgnoreCase ”  + s2 + “---> “ +
s1.equalsIgnoreCase(s3));
        }
}
```

The output of the program will be :

Good equals Good ---> True

Good equals gOOd ---> False

Good equalsIgnoreCase gOOd ---> True

regionMatches() : This method compares a specific region inside a string with another specific region in another string. Its general form is :

boolean regionMatches(int *startIndex*, String *str2*, int *str2StartIndex*, int *numChars*)

An overloaded version of this method allows you to ignore cases in the comparison. Its general form is :

boolean regionMatches(boolean *ignoreCase*, int *startIndex*, String *str2*, int *str2StartIndex*, int *numChars*)

In both these methods, *startIndex* specifies the index at which the region begins within the invoking **String** object. *str2* specifies the String being compared. The index at which the comparison should start within *str2* is specified by *str2StartIndex*. *numChars* gives the number of characters to be compared. In the overloaded method, if *ignoreCase* is **true**, the case of characters is ignored. Else case is considered.

startsWith() and **endsWith()** : The **startsWith()** method determines whether a given **String** begins with a specified string, and **endsWith()** method determines whether the String ends with the specified string. The general forms of these methods are :

boolean startsWith(String *str*)

boolean endWith(String *str*)

str is the **String** to be tested. The method return **true** if the string matches, else it returns **false**.

eg. MyString.startsWith(“My”) returns **true**

MYString.endsWith(“ing”) returns **true**

It is also possible to provide a starting point in the **startWith()** method as follows:

boolean startsWith(String *str*, int *startIndex*)

where *startIndex* is the index at which point the search should begin. eg.

MyString.startsWith(“Str”, 2) returns **true**

compareTo() : In many applications you need to know, whether a string is less than, equal to or greater than another string. A string is less than another string, if it comes before the other in dictionary order. A string is greater than the other string if it comes after the other in dictionary order. For this comparison the method is **compareTo()**. Its general form is :

int compareTo(String *str*)

where *str* is the **String** being compared with the invoking **String**. This method returns an **int** value, and the result is interpreted as follows :

Value	Meaning
Less than zero T	he invoking string is less than str
Greater than zero	The invoking string is greater than str
Zero	The two strings are equal.

The version of the method, to be used if you wish to ignore case is :

int compareToIgnoreCase(String *str*)

8.5 Check Your Progress.

1. Write True or False.

a) In Java, it is possible to perform a comparison which ignores the case.

b) A string is less than another string, if it comes before the other in dictionary order.

c) It is possible to determine whether a given **String** begins with a specified string.

8.6 SEARCHING STRINGS

You can search for a specified character or a substring within a given string with the following methods :

indexOf() - searches for the first occurance of a character or a substring

lastIndexOf() - searches for the last occurance of a character or a substring

The methods return the index at which the character or substring is found, else return a -1 if no match is found.

The general forms of the methods are :

int indexOf(int *ch*) - to search for the first occurance of a character

int lastIndexOf(int *ch*) - to search for the last occurance of a character

where ch is the character to be searched.

eg. tooth.indexOf('t) will return 0

tooth.lastIndexOf('t') with return 3

To search for the first or last occurance of a substring we use the overloaded form of the method as:

int indexOf(String *str*)

int lastindexOf(String *str*)

where *str* specifies the substring

For both these forms, it is possible to specify the starting point of the search using the following forms:

int indexOf(int *ch*,int *startIndex*)

int lastIndexOf(int *ch*, int *startIndex*)

int indexOf(String *str*, int *startIndex*)

int lastIndexOf(String *str*, int *startIndex*)

where *startIndex* specifies the index at which point the search begins. Remember that for **indexOf()** the search runs from *startIndex* to end of string, whereas for **lastIndexOf()** the search runs from *startIndex* to zero.

8.7 MODIFYING A STRING

Whenever you want to modify a string, use one of the following string methods.

8.7.1 substring

Using **substring()** you can extract a substring from a string. It has two forms. The first form is :

String substring(int *startIndex*)

here *startIndex* specifies the index at which the substring will begin. This form returns a copy of the substring which begins at *startIndex* and ends at the end of the invoking string.

In the second form, you can specify both the starting index and ending index of the substring :

String substring(int *startIndex*, int *endIndex*)

where *startIndex* specifies the beginning index and *endIndex* specifies the stopping point. The string returned will contain all characters from the beginning index, upto but not including the ending index.

eg. String s1 = “Mystring”.substring(2) will assign the substring "string" to s1.
String s = “Mystring”.substring(2,5) will assing "str" to s

8.7.2 concat

This method is used to concatenate two strings and has the following form :

String concat(String *str*)

This method creates a new object, that contains the invoking string and the contents of *str* appended to it at the end.

eg. String s2 = “New”.concat(“Strings”) will put "NewStrings" into s2.

concat() performs the same function as +.

8.7.3 replace()

This method replaces all occurances of one character in the invoking string with the specified character. Its general form is :

String replace(char *original*, char *replacement*)

The method returns the string, where the characater specified by *original* is replaced by the character specified by *replacement*.